

HANDBOOK ON ***APPLICATION PROGRAMMING*** ***INTERFACES (APIs)***

CONTENTS

1. What is API?	02
2. Why APIs?	03
3. Key entities in API Ecosystem	03
4. API Management	04
API Build & Deploy	04
API Manager	05
API Portal	06
API Gateway	07
API Analytics	08
5. API Governance	09
Business Governance	09
Technology Governance	13
6. Security Considerations	14
Attack Landscape	14
Mitigation Approach	16
Authentication Techniques	17
Authorization	18
Confidentiality and Integrity	19
Non-repudiation	20
7. API from Standard Entities	21
Annex: I	22
API Risk Evaluation Guideline	22
Security Recommendations Checklist	23
Process for Publishing API	24
Approval Matrix for Publishing API	25
Partner Risk Evaluation	26
Partner On-boarding RACI	27
Annex: II	28
API Technology Management – Responsibilities Matrix	28
Annex: III	29
API Security Vulnerability List	29
API Security Assurance Checklist	31
Token Threat/Vulnerability and Mitigation	33
Glossary	35
References	36

Foreword

API Revolution and Open Banking Evolution

THE hallmark of digital transformation of any bank is being able to do business with customers, wherever they are, across various transaction modes or interaction types. Today everything that goes into making a bank digital like delivering new customer experience, building software ecosystem and moving to multi-cloud environment involves APIs.

APIs (Application Programming Interfaces) are becoming, if not already become, the de facto standard for building and connecting applications. In simple terms, APIs let software talk to other software. API-based architectures are designed for decentralized teams, very lightweight governance, and iterative, fast development.

API-first banks often release APIs quickly in the form of minimum viable products and then use analytics to iteratively and continuously improve the APIs. As APIs abstract backend complexity behind a consistent interface, they decrease the understanding of backend systems that a developer needs to possess. This shift democratizes access to digital assets, making it easier for developers to leverage systems and data. The success of UPI (Unified Payments Interface) is to a large extent attributable to its API approach.

Banks have understood that APIs are not just integration technology to connect applications and data but as software products that empower



developers to unlock new digital business models and opportunities. Indeed, the adoption, design, and management of APIs are increasingly driven not by IT middleware requirements but by strategic business goals relevant to senior and top level management of banks.

The focus of banks is on Open Banking, which is a major source of innovation in the banking industry. Open Banking relies heavily on APIs. It is threatening long-established banking practices and procedures. And banks are converting the threat into a great opportunity.

It is in this background the IDRBT Team has attempted to put together all relevant information on APIs and the current handbook is the result of the effort. I am sure the handbook serves as a good reference material for banks and software companies that are working with banks in development of APIs.



(Dr. A. S. Ramasastry)
Director, IDRBT

Date: August 20, 2019

Place: Hyderabad

1. What is API

An Application Programming Interface (API) is a set of clearly defined methods of communication between software components without any user intervention. API consists of a set of instructions and standards to be followed by the participating applications. APIs operate on an agreement of inputs and outputs and are independent of any specific programming language.

One can view an API as:

- ★ A means of requesting and receiving information from a system/server, so that

it can then be presented to the user in a context/process-specific way, e.g. in a mobile app or web application.

- ★ A way of de-coupling system-to-system interactions.
- ★ A contract between the application provider and the consumer describing, which information or functionality they can gain access to and how that access is provided.

APIs are commonly categorised as shown in the diagram below:

API Category	Private 	Partner 	Public/Open 
Description	API used internally to facilitate the integration of different applications and systems used by a company	API used by known business partners, with some special contractual relationship.	API used by anyone who meets the organisation's access control policy
Advantages	▪ Rationalized Infrastructure ▪ Reduced costs ▪ Increased flexibility: "real-time" business ▪ Improved internal operations	▪ Value-added service ▪ Up-sell	▪ Delegated R&D ▪ Increased reach, Traffic ▪ New revenue stream
Example	An API that provides CIF information to other applications	Trading platforms communicating with banking applications	Unified Payments Interface

2. Why APIs?

Organizations across industry sectors are competing for customers based on their ability to provide a superior user experience. This is relevant for banking amid the emergence of digital banking applications that continue to corner the market in terms of consumer convenience and choice. The customers are also looking for better, more advanced technology that facilitates fast and secure payment platforms and various other user experiences.

This in turn has positioned FinTech firms as most-likely-to-succeed when it comes to creating positive banking experiences. However, by working together and incorporating the advantages of well-developed APIs that are tailored to the future of payment solutions, banks and fintechs can maximize their strengths and take the customer experience to an even higher level than either entity could do on its own.

3. Key Entities in API Ecosystem

API Owner

The entity or business function that is the ultimate owner of a given API. API owner is responsible for the underlying governance function and is also responsible for defining, ensuring and chartering the API Provider.

API Provider (Producer/Publisher/Creator)

The entity that is responsible for implementing the software and exposing its API for others to use. API providers have the right to:

- Define and publish terms of use (including SLA) to restrict the way the API is consumed
- Modify the implementation of the API at any time, as long as the new implementation complies with the API definition and the agreed SLAs are met

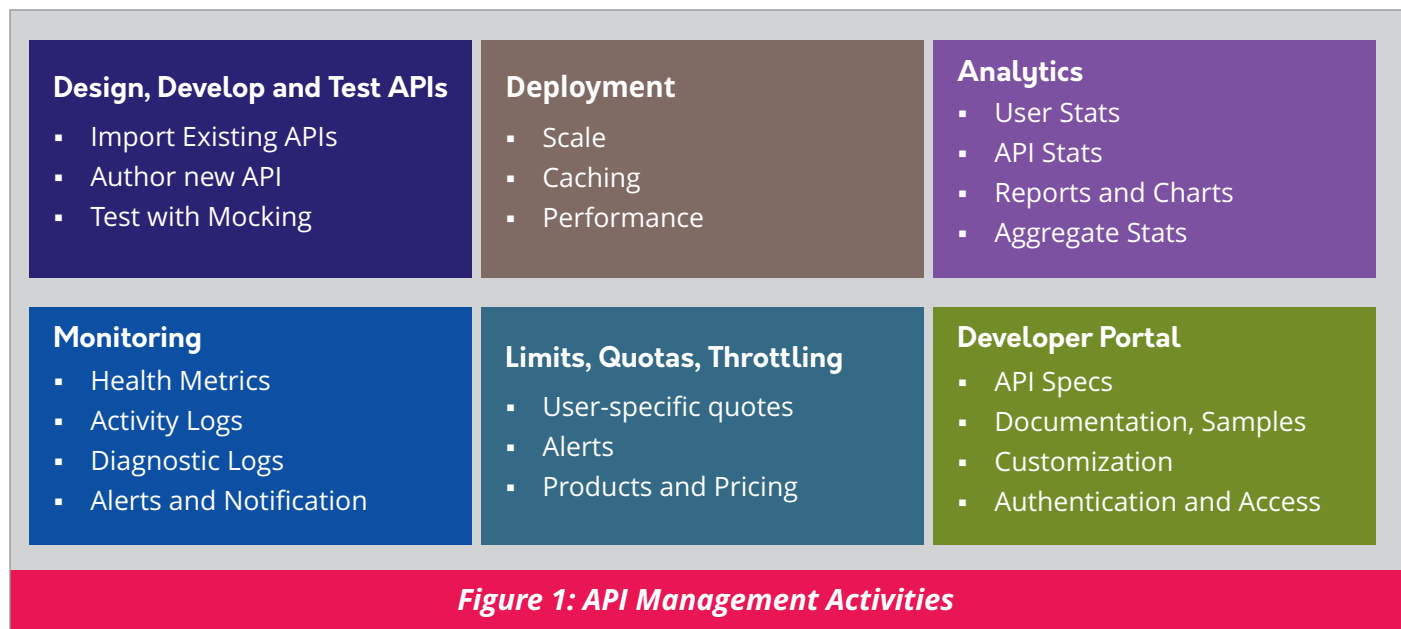
In certain cases, the API Owner and Provider can be the same entity.

API Consumer

API consumers are entities, which consume the published API in compliance with the terms of use, subject to authentication and authorization under which the API use is permitted. API consumers can rely on the published API definition without worrying about the specific implementation details (including implementation platform, internal architecture, etc.).

4. API Management

API management is the process of creating and publishing web APIs, enforcing their usage policies, controlling access, nurturing the subscriber community, collecting and analyzing usage statistics, and reporting on performance as categorized in Fig 1. API Management components provide mechanisms and tools to support developer and consumer community.



4.1. API Build & Deploy

An API should deliver the functional and non-functional requirements such as Security, Usability, Reliability, Testability and Scalability, which should be addressed in the build and deployment phases. Building API involves:

- ★ **Modelling:** Visually or programmatically specify the data needed for the API endpoints
- ★ **Orchestrating:** Combine and normalize data from multiple sources
- ★ **Transforming:** Convert legacy formats (e.g. XML) to modern consumable formats (e.g. JSON)

- ★ **Documentation:** Auto-generate docs and code-snippets for models and API operations.

API Design Principles

- ★ **Technology Agnostic:** The API design is agnostic to applications, programming languages, and platforms, and aims at seamless and secure flow of electronic data across different applications.
- ★ **Reliability and Scalability:** Non-repudiation, consent, digital signatures, logging requirements bolster reliability and accountability of the dataflow during API interaction. Asynchronous mechanisms and callbacks improve scalability of the system.

- ★ **Privacy by Design:** Customer information should be protected from abuse and compromise. The design of API should ensure the privacy of customer data ground-up through audit trails and consent driven framework that guarantees non-repudiation.
- ★ **Security by Design:** The API should be designed from the initiation to be secure. There should be end-to-end security of data (PKI, Digital Signature Certificates, tamper detection) and it should be network agnostic and data centric.
- ★ **Minimalist and Evolutionary Design:** The API design should be simple and minimalistic. It should not present adoption barriers for the ecosystem.
- ★ **Customer Centric:** The customer experience and ease of use are critical to successfully delivering the API for developing the system. The design principles take into account the various stakeholder responsibilities and mechanisms to simplify interactions and minimise friction.
- ★ **Open APIs for Interoperability and Layered Innovation:** People and systems should have programmatic interfaces for sharing and accessing the information available to them. The specification defines the standard APIs to promote interoperability and deliver services that are designed to work with any device, any form factor, and any network.

- ★ **Transparency and Accountability through Data:** Aggregated metrics shall be made available via APIs for transparency. The access to aggregated metrics will ensure high-quality analytics, accurate fraud detection, shorter cycles for system improvement and, most importantly, high responsiveness to customers' needs.

4.2 API Manager

API Manager generally includes an API portal, API gateway with support for API monetization and analytics capabilities to help a business make the most out of their API program. Key responsibilities of API Portal and API Gateway can be seen from the Fig 2.

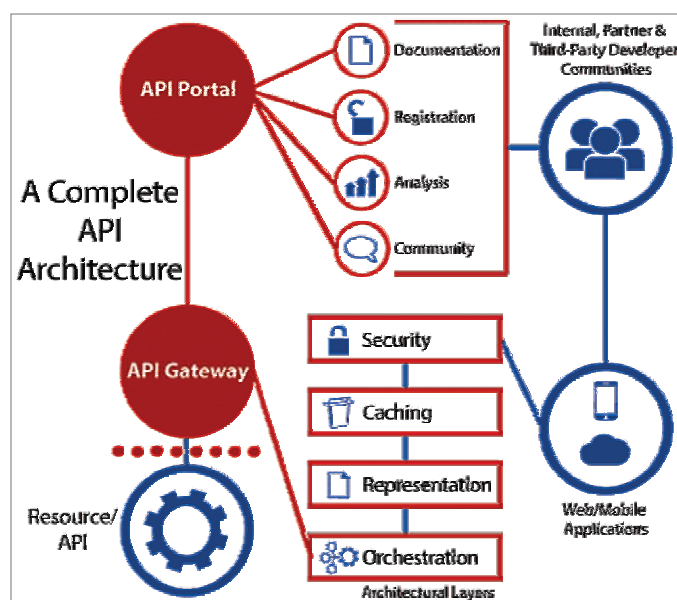


Figure 2: API Manager Architecture

4.2.1 API Portal

The API Portal is a technical layer enabling a bank to control an API's visibility and behaviour. It is usually exposed as a UI/console to internal staff only as an administration component and a development portal to internal or external developers. It offers:

Management Portal

- ★ **API Catalogue Administration:** Provide governance over the API catalogue, what is exposed and what is not, what is available for external use or internal use only
- ★ **QoS Policy Definition:** Administrators use the API management layer to define QoS policies such as defining quota limits for an application or API.
- ★ **API Registration:** Administrators register APIs in the API manager to expose them to application developers. As part of the release process, API administrators register APIs in a management layer.
- ★ **Identity Management:** Issue and manage API credentials.
- ★ **Access Control:** Administrators can control API access by defining access policy for the API gateway to enforce, disabling an API key or other credential that is being used maliciously.
- ★ **Support:** Provide customer support for inquiries, incidents and problems.
- ★ **Monetization:** Monetize an API marketplace and billing processes.

- ★ **Reporting & Analytics:** Tools to visualize and navigate API statistics and usage data.

API Developer Portal

Banks need to be able to engage, onboard, educate and manage consuming application developers, whether inside or outside the organization. The easiest way of making these capabilities available to application developers is to offer a dedicated website or development portal with the following capabilities to the consumer:

- ★ **Discovery:** Make it simple for developers to find APIs, which match their need.
- ★ **Provide Self-service Access:** For developers to browse APIs, their attributes' documentation and an easy try-it feature to a sandbox environment.
- ★ **On Boarding:** Allow developers to register and sign-up for API usage plans with support for automatic approval or manual approval workflows
- ★ **Knowledge:** Provide knowledge resources such as an API specification, documentation, code samples and developer community to share knowledge, experience and best practices via a support community such as online forums
- ★ **API Evaluation:** Give developers a "sandpit" capability to try APIs interactively
- ★ **Metrics:** Deliver insight into API usage and performance
- ★ **SLAs:** Publish API SLAs, define performance, availability metrics, etc.

4.2.2 API Gateway

The API Gateway is the means through which an organisation offers APIs to the outside world. This component (physical or virtual) hosts the APIs ready for consumers to access. It provides the bank with the ability to control who has access to their APIs by enforcing policies for access control. Some API gateways also offer additional capabilities such as:

- ★ **Security:** Offering authentication and authorisation services. The API Gateway is essentially the Policy Enforcement Point (PEP)
- ★ **Access Management:** Securing access to the API with practices such as authentication, authorization and session management.
- ★ **Information Security:** Defending the API from information security threats. Uses threat protection capability to protect backend systems against common vector attacks such as SQL Injection, Cross Site Scripting and Cross Site Forgery
- ★ **Availability:** Managing the API to provide high availability
- ★ **Capacity Management:** Planning to meet demand for the API with sufficient resources such as licenses and computing infrastructure
- ★ **Performance:** Maximizing API efficiency to support consuming applications and minimizing downtime
- ★ **Audit Trail/Logging:** Recording an audit trail. Saving events as messages in a file or database for later analysis and auditing
- ★ **Data Transformation:** Converting transiting data into application-friendly formats
- ★ **Orchestration:** Composing new, aggregate APIs from multiple existing APIs
- ★ **Quality of Service (QoS):** Provides QoS for consuming applications and enables a bank to apply policies for a specific consuming application or policies that protect back-end systems from overload. QoS is implemented through API Traffic Management
 - **Caching** to improve API and App performance by storing data from backend resources in a cache, from where they can be retrieved quickly
 - **Quotas and Rate Limits** to limit the number of connections apps can make via the API to the backend. Quotas are more about ensuring consumers are sticking to their end of the API contract, typically based on a service-level agreement (SLA) like number of calls they can make to an open API per second, minute, hour, etc. This is also where monetizing an API comes into play
 - **Rate Limits** are all about controlling big bursts of traffic to API. They keep track of calls to an API during a period of time, based on username, API key, or IP address
 - **Spike Arrest Capabilities** to protect backend systems against severe traffic spikes and denial-of service attacks.

Spike arresting is a built-in braking system that shuts access down if calls get out of control - from either malicious intent or just poorly written code

- **Throttling** is a bit different, but also deals in restricting calls to an API. Instead of receiving an error message, consumers' calls to the API are slowed down if a certain number of calls are exceeded in a set time period.

4.2.3 API Analytics

API analytics provide real-time insights into the business and optimize the delivery and value of APIs. They leverage the collected API data to generate predictive analytics dashboards analyzing trends and outliers.

- ★ **API Usage Trends:** Segment the audience by top developers and apps; understand usage by API method to know where to invest; create custom reports on business or operational-level information.
- ★ **Real-time Monitoring:** All the information is gathered, analyzed, and provided immediately.
- ★ **Analytics Answered Questions:** Which API methods are most popular? How much API capacity will be needed next year? Why is the API down?
- ★ **Predictive Analytics:** Understand customer behavior across all digital channels; combine both profile and behavioral data to predict the next best action; and turn prediction into action with batch and real-time APIs for all digital channels.

Some Examples:

Category	Description
At A Glance	To see at a glance – Blocked Calls, Successful Calls, Failed Calls, Average Response Times and Bandwidth. These can be broken down by Top Developers, Top Products, Top APIs, Top Subscriptions and Top Operations
Activity	Shows Blocked Calls, Successful Calls, Failed Calls, Average Response Times and Bandwidth broken down by APIs, Developers, Products, Subscriptions, API and Operations. Filter by Date Range, Region, Product, APIs and Operations to be available
Health	Shows for each Operation: Average Response Time, Service Time and Caching stats. One should be able to filter by Date Range, Region, Product, APIs and Operations
Usage	Shows for each Operation: The number of calls and bandwidth consumed. One should be able to filter by Date Range, Region, Product, APIs and Operations

5. API Governance

API governance enables consistency across APIs, thereby saving time and money. It ensures that components are reused, and that APIs are built proactively to achieve specific goals and bring value to the business. API governance also helps companies make intelligent decisions regarding API programs and establish best practices for API lifecycle management - building, deploying, consuming & retiring APIs. API Governance covers both business and technical areas.

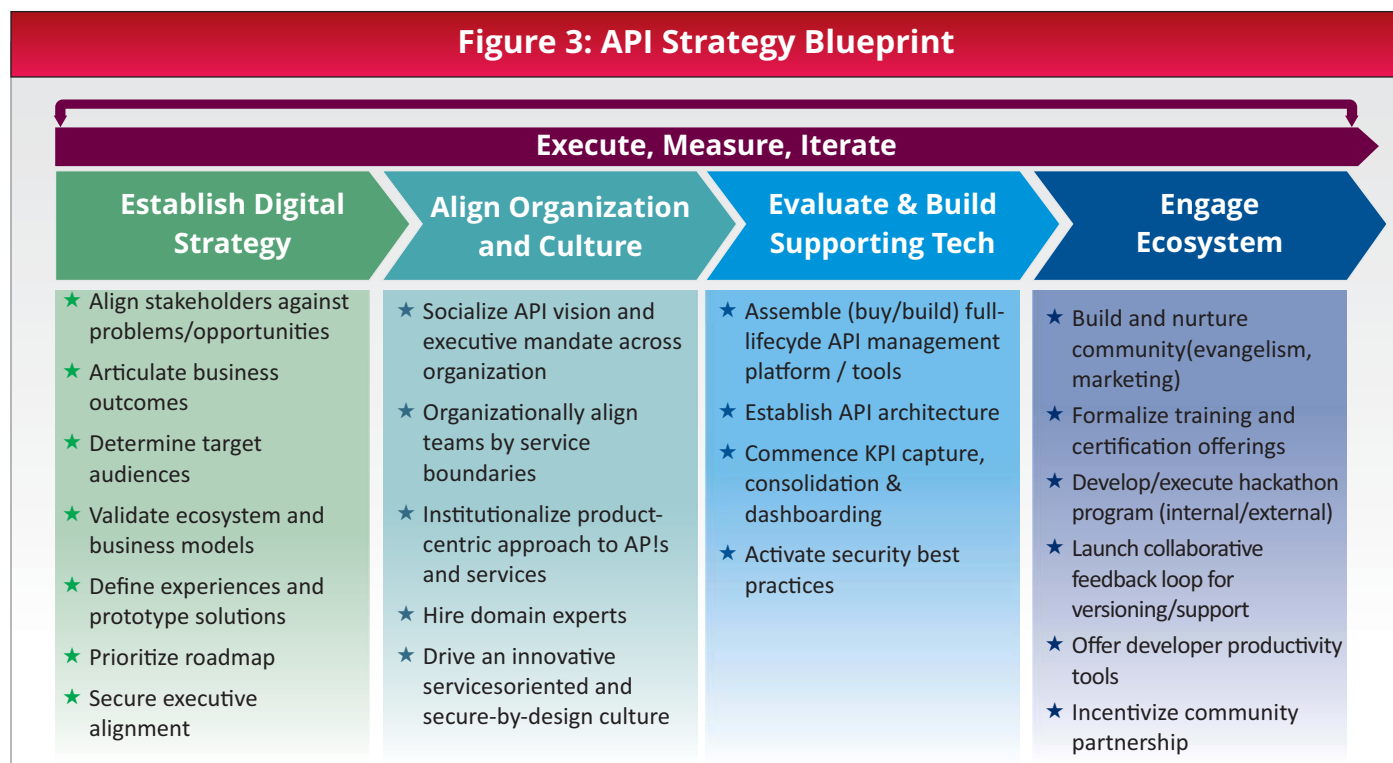
5.1 Business Governance

To get initiated into API, one needs a strategic approach to better support their internal business and have easier integrations with partners. API strategy should consist of public,

partner and private APIs. When a bank's business releases public APIs that power consumer-facing applications, it enables new ways to engage and connect with its customers through web, mobile, and social apps.

By developing private APIs, banks can offer their employees and partners, new tools that help them streamline operations and serve customers even better. In this dynamic environment — as more and more businesses create and incorporate APIs — it is increasingly critical for innovative businesses to develop and execute successful API strategies. The strategy for API can be formed following the steps described in Fig 3.

Figure 3: API Strategy Blueprint

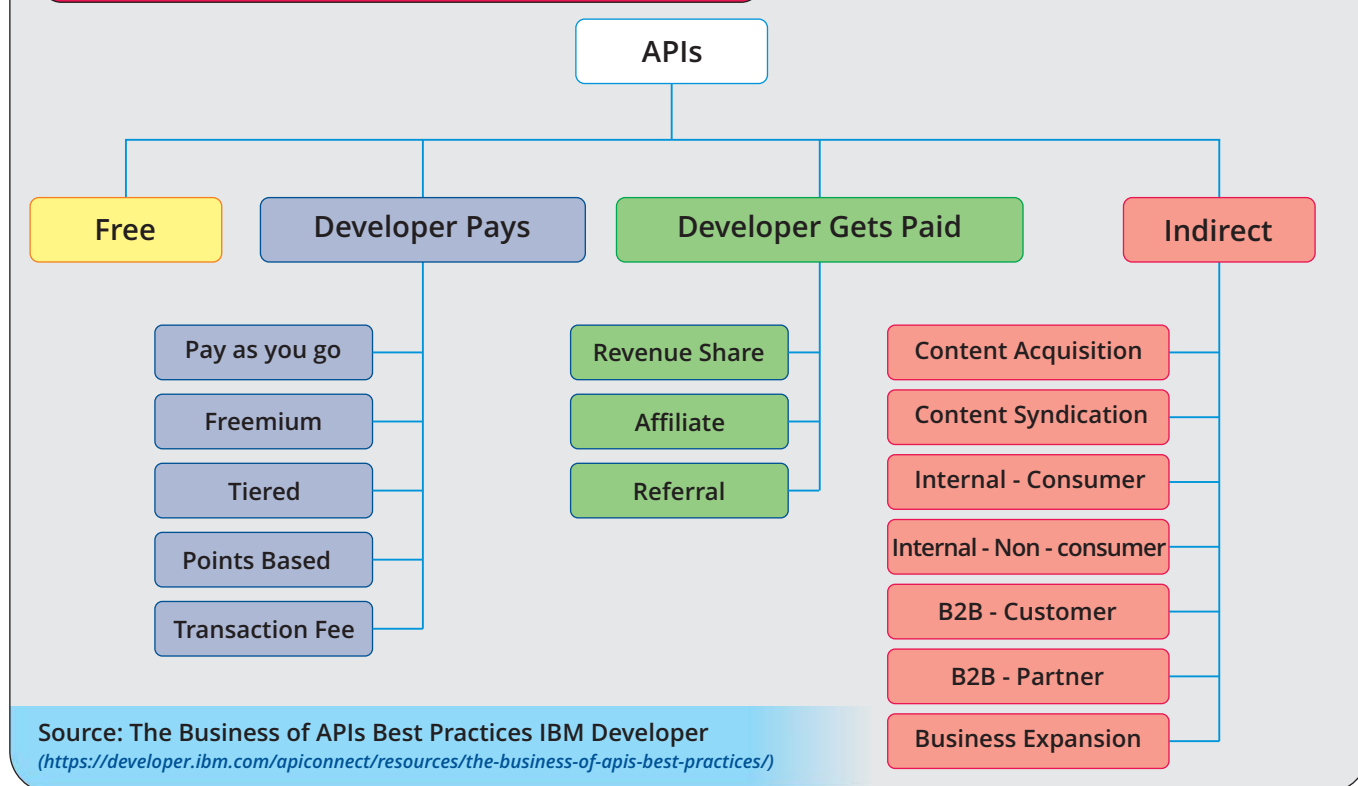


Source: <https://www.mulesoft.com/resources/api-strategy>

From a business perspective, one should consider the following topics.

- ★ **Centralization:** A central group consisting of business and technology governance, that creates and enforces policies related to API governance across the organization. This helps in standardization of API Designs, which ensure that all APIs built remain consistent. One way to standardize API design is by establishing and enforcing design guidelines for all APIs.
- ★ **API Identification:** One of the most important aspects. Creating compelling consumable APIs goes a long way in driving success. Putting appropriate roles, organization, and methodology to identify good APIs is extremely important. In addition, understanding the API portfolio and a good attempt to drive reuse are also helpful.
- ★ **Automation:** API contracts, documentation, and tracking are processes that can be automated and should be part of overall API governance. There are many tools available that automate API design and governance processes.
- ★ **Versioning:** Creating APIs that are consumer-oriented versus provider-oriented has many benefits, including a higher likelihood of backward compatibility. Having a good versioning strategy and focusing on creating backward compatible APIs can go a long way in ensuring the success of API initiative. If APIs are frequently seen to be non-backward compatible, consumers will believe that the provider does not know what it is doing. Eventually they will look for other API providers who do not cause them so much rework. Planning for backward compatibility allows moving implementations forward in a less disruptive manner.
- ★ **API Access (Security):** Establishing who can access each API (business asset) and setting up the visibility and security enforcement around the API. More on this is in Section 6.
- ★ **Entitlement and Enforcement:** Defining and enforcing quotas and rate limits.
- ★ **Monetization:** This entails deciding upon the business model, setting up the monetization capability required and measurements. Few options are detailed in Fig 4.

Figure 4: API Monetization Options



- ★ **Privacy:** Ensuring consumer privacy. Apps need to be validated to be sure that they are allowed to access the API. User identity for the App user should be secured so that the App itself does not have access to the user's credentials and User identity should pass into the systems of record to ensure only that user's data is accessed. In addition, ensuring organization privacy is important. Organizations that consume the same API may be competing with one another and should not be able to see another organization's customers or data.
- ★ **Legal:** Ensuring proper usage of corporate assets, especially when assets are being used by other businesses. The

guidelines and policies are to be defined keeping the below aspects in consideration:

■ Data Security

- ✦ Does the provider own the data being provided?
- ✦ Are all audiences entitled to access this data?
- ✦ What rights is the provider granting the consumer of the API to use the data provided?
- ✦ What is the required policy for data retention?
- ★ Relevant clauses to protect sensitive personal data or information
- ★ How does the provider communicate the terms of use to the API consumer?

- ★ What requirements does the provider have for attribution of the content or use of its brand? Does it need to give attribution to some other entity?
- ★ How will the provider find out and deal with consumers who do not use the API appropriately?
- ★ What are the liabilities of the provider?
 - Security for API Key, Secret, Digital certificate, etc.

★ **Regulatory Requirements**

- To comply with the provisions of the Information Technology Act, 2000 and the applicable rules thereunder, including without limitation the Information Technology (Reasonable Security Practices and Procedures and Sensitive Personal Data or Information) Rules, 2011;
- Comply with all notifications, guidelines, circulars, issued by the RBI

★ **Governance**

- Rights for Inspection and Audit by the API provider
- Partner acceptability to upgrade to new version with defined timelines

- ★ **Measurements:** Establishing success criteria, metrics to be collected and implementation of the gathering and communication of these metrics.

- ★ **Deprecating:** An effective version control process helps deprecating APIs much easier. An effective deprecation should specify what the replacement API is or

how the existing business function will be impacted and gives time to developers to modify their functions. Post deprecating, once the traffic to the API stops, the API can be retired.

Maturing Business Governance Process

API initiatives start with focus on internal consumers (own employees or contractors), then move to partner models, and finally to public API consumers. As bank's API initiative matures and additional audiences are added for APIs, they become less and less under control and more considerations need to be added for governance processes. It is recommended that banks learn and iterate governance along the way and address critical concerns early while adding necessary enhancements later.

- ★ **For Internal Audiences:** During the beginning, efforts to think about API Identification security is internal, so it is a time to get comfortable with the capabilities and implementation. Versioning considerations can begin but again will improve with experience. Monetization, if any, is usually a charge back model for internal. Entitlement/enforcement is usually soft – notification of usage beyond a limit is all that is needed. Gathering and communication of metrics begins.

- ★ **For Partners:** API Identification, Security, and versioning are firmed up. Monetization might begin to occur, and entitlement/enforcement could be either soft or hard. In addition, concerns about privacy need to be addressed.
- ★ **For Public:** All the above, plus more focus on monetization and stricter entitlement enforcement. Legal concerns are important now.

Annex: I provides details on API Risk Evaluation Guideline; Security Recommendations Checklist; Process for Publishing API; Approval Matrix for Publishing API; Partner Risk Evaluation and 'Partner On-boarding RACI.

5.2 Technology Governance

Here's a quick look at some of the technical governance considerations:

- ★ **API Standards/Best Practices/Naming Conventions:** Setting up these criteria is often done early. For example, SwaggerHub provides tools for standardizing design styles and managing common models
- ★ Set up best practices around granularity and simplicity

- ★ **Lifecycle:** Few stages should be required to ensure speed of delivery
- ★ **Deployment/Publishing:** Early environments are usually handled synchronously. However, production is usually isolated and requires a disconnected deployment approach (usually scripted)
- ★ **Security:** Setting up technical considerations around security, e.g. App security handled via App keys and secrets. Consumers of the API may be identified using OAuth to shield their private login information from the App itself
- ★ **Scale:** API entitlement levels are used to plan for capacity, the gateway enforces these entitlements and shields the systems of record from too many requests. Analytics can help plan for increased requirements on systems of record
- ★ **Integration:** Understanding the difference between APIs and Services and creating criteria for these is critical.

Annex: II provides 'API Technology Management – Responsibilities Matrix'

6. Security Considerations

All these years, organisations used to expose a web interface, with good control over the information released via that interface. In contrast, APIs offer direct, machine-to-machine access to resources and information, which makes it less obvious when information is released in a wrong way. It is important for internal business stakeholders to decide what information and resources to release via this channel, i.e, API, and to whom.

6.1 Attack Landscape

1. Business Logic Attacks (BLA)

BLA exploit flaws mostly present due to functional level complications & restraints, managing the exchange of information between a user interface and the application's supporting database. Programming flaws may also contribute but due to logical anomalies rather than syntactical errors.

API enables business-related operations available as procedure calls, making it easier to attack the business logic of a company using an API attack. These attacks include legitimate input values, thus detecting the attack is difficult. BLA's abuse the functionality of the application. Examples of BLA's are:

- ★ Tampering authentication flags and privilege escalations
- ★ Generating fraudulent transactions by exploiting business constraints or bypassing business logic
- ★ Parameter tampering
- ★ Cookie tampering and business process/

logic bypass

- ★ Exploiting clients' side business routines embedded in JavaScript, Flash, or Silverlight
- ★ Identity or profile extraction
- ★ LDAP parameter identification and critical infrastructure access

2. Integration with Third Parties

Catering to third party API with required environment as per our requirements is not always feasible. Additional costs are required to fix these issues, apart from the purchasing costs. There may be less control over API lifecycle, as there could be few sections, which client may want to overlook. Working on those aspects might outweigh the benefit of going for 3rd party integration.

Moreover, since client would be integrating its products with 3rd parties, the security of client's products and services completely rely upon the safeguards, security measures followed by the 3rd party API. There could be inherent incompatibility issues, fixing which on integration would require cost/benefit analysis. In addition, integrating 3rd party with client complex legacy system infrastructure should be well thought out, based on feasibility and business functionality.

3. Compliance & Regulatory Issues

APIs are expected to help banks meet new regulatory requirements around the world. The PSD2 Directive introduced compliance requirements for banks and other financial institutions, including the enforcement of new security requirements and interoperability standards aimed at reducing barriers to entry

for nonbank card and internet payment providers.

APIs are essential for regulation and compliance, and for leveraging big data. Few regions have open regulatory standards, while others mandate regulatory behavior. Compliance risk has become one of the most significant ongoing concerns in the BFSI domain. Customers, partners opt to look for moral bankers, failure to abide by the same leads to loss of business, reputational risks, and financial impact.

4. Data Sharing Issues

Data exchange should ensure proper authentication and authorization. Cases of unauthorized access and failure to follow required levels of consent, may lead to data leakage, confidential data breach. This disrupts the business process leading to operational loss.

5. Interfacing with Legacy Core Banking Systems

Considering technical compliance overheads, to provide requested responses after customer inputs details, banks need to create TTP-facing, open access front ends, overcoming technical challenges in the back-office integration of legacy core banking systems. If the integration, however fails, it would be of no/little value to banks and banks would fail to leverage hold on the customer data details. Also, if the data held in the database goes underutilized, the ineffective usage of technology may lead to limited access to refined customer data.

6. Issues in Managing Digital Identities

The pace at which the digital environment is growing, managing digital identities has become a major problem. As multiple agencies are being integrated for providing a service, data sharing issues and data privacy issues are looming large. Due to high face value, financial data has always been a high-profile target for hackers. Risk in digital identity management increases as the number of integrations increase. The attack surface also increases exponentially.

API Security Risks

The security risks that APIs introduce will be similar to the traditional risks experienced on any web channel (websites and web applications), except there is:

- ★ Increased attack surface due to multiple services/entry points that can potentially be exploited
- ★ Back-end data, back-end architecture and back-end applications exposed inadvertently
- ★ Loss of integrity, confidentiality and availability of data exposed through API
- ★ Allowing access to more information than was intended, due to unknown loopholes while retrieving API resources (especially if fields requested are built straight into a DB query)
- ★ Polluting data stores, feeding misinformation into a system due to improper write operations
- ★ Denial of Service Attack by overloading

the server or data store due to improper write operations

- ★ Use of wildcards in search fields can shut down APIs and back-end applications
- ★ Cross site scripting attacks made possible by consuming applications not checking user inputs
- ★ SQL injection into consuming applications which cause database damage at the API backend
- ★ Parameter attacks such as HTTP Parameter Pollution (HPP)
- ★ Man-in-the-middle attacks, modifying API requests or responses leading to data eavesdropping or misinformation insertion
- ★ Subverting authentication or authorisation mechanisms to spoof messages from legitimate consumers
- ★ Stealing authentication tokens to obtain information illicitly
- ★ System information leakage through API error messages revealing details about an API's construction or underlying system makeup
- ★ Broken Session IDs, Keys and Authentication create exposure to unauthorized access through authentication factors that are not functioning because of poor security design or technology bugs.

6.2 Mitigation Approach

API risks need to be mitigated in a number of ways. There is no single off-the-shelf security solution, which can address all aspects of API security. APIs need to be secure by design; security should be built in from initiation and

be considered within the context of existing protection mechanisms.

Key principles that should be applied when designing API security frameworks:

- ★ Design with the objective that the API will eventually be accessible from the public internet, even if there are no plans to do so at the moment.
- ★ Use a common authentication and authorisation pattern, preferably based on existing security components: avoid creating a bespoke solution for each API.
- ★ Least Privilege: Access and authorisation should be assigned to API consumers based on the minimal amount of access they need to carry out the functions required.
- ★ Maximise entropy (randomness) of security credentials by using API keys rather than username and passwords for API authorisation, as API keys provide an attack surface that is more challenging for potential attackers.
- ★ Balance performance with security with reference to key lifetimes and encryption / decryption overheads.

The main areas that API Security covers are:

- ★ **Authentication and Authorization:** The consuming application is known and can only get access to API resources they are allowed to
- ★ **Confidentiality:** Message content is not been tampered with between consumer and provider
- ★ **Integrity:** Resources are reliably from the provider intended when the consuming

application made the request

- ★ **Availability and Threat Protection:** The API is available when needed, and not brought down by attacks from malicious consuming applications. Quarantine/ isolate misbehaving consumers.

6.2.1 Authentication Techniques

Authentication is the process of verifying the identity of a customer (or device) who presents identity credentials and authentication key(s). Banking organisations mainly perform the following type of interactions, where participating entities communicate with each other:

- ★ **Bank to (Internal) Bank:** In this pattern, an API is developed for internal use only by bank applications/systems. The authentication mechanism needed to validate the internal application, and implement protection between the internal application and the API on the API gateway.
- ★ **Bank to Customer:**
 - **Bank to Application Developer:** When an API is released for external use, the first interaction will be with App

Developers who want to try the API out. This will normally be via the API Developer Portal. The Application Developer should be authenticated to the API Developer Portal to register their new application and attain the relevant credentials, which are used to secure interactions with the new application during development. The developer has to agree to the conditions of use and subsequent usage of the API can then be traced via the API keys

- **Business-to-Business (B2B):** The system-to-system model is where an API is being used to enable information sharing or integration with an external system e.g. a partner bank/entity gaining access to supporting information. In this model, the aim is to ensure that only the correct consuming system has access to the API, and that the API is protected from malicious use. B2B models often carry sensitive information, so the consuming system should be authenticated to the provider for authorised access, confidentiality and integrity.

Mechanism	Purpose/When to Use
Anonymous	Not recommended
Username and Password	Not recommended for public facing API, can be used only for internal development and training purposes
API Keys	For application authentication, i.e., STP
Certificates	For all B2B communication
OAuth2.0 and security tokens	For internal and external production API
SAML	Wherever SAML provider is available
OpenID	Wherever OpenID provider is available

Best Practices for API Authentication

- ★ The authentication model should be protected against typical API vulnerabilities and threats, as listed on OWASP (Open Web Application Security Project).
- ★ TLS (formerly known as SSL) encryption should be used for authentication otherwise the username and password combination can be easily decoded.
- ★ User credentials and sensitive data should be handled securely
- ★ HTTP basic authentication can be used for debugging and getting started scenarios but should not be used in production.
- ★ API Keys should be used instead of traditional username/password authentication.
- ★ All communication should be over TLS, to protect the API key in transit.
- ★ API keys in headers are better than API keys in URLs since it will keep the URL constant for all consumers and it does not expose API keys when sharing links.
- ★ It should be possible to revoke API keys in case they are compromised in some way.
- ★ Implement IP/Port whitelisting/VPN/Leased line on basis of traffic volume. Application level firewalls can be used to enforce the mapping of API keys to IP/Port/VPN/Leased line.
- ★ Centralised user management mechanism should be used. Absence of centralised user management mechanism may lead to undocumented user creation/modification.

- ★ Standard methods to sign should be used. Proprietary methods for signing request are prone to design errors as it is easy to get the signature mechanism wrong - and thus accidentally making the signature technique easy to circumvent.
- ★ An application could use multi-factor authentication (MFA) to enhance other authentication techniques to mitigate identity risk.

6.2.2 Authorization

Once a user is authenticated (e.g. using username and password), an authorisation process will grant (or deny) them the right to perform an action or access to information. Normally this authorisation process is applied using either a coarse-grained or fine-grained access control process. Authorisation is the act of performing access control on a resource. Authorisation does not just cover the enforcement of access controls, but also the definition of those controls. This includes the access rules and policies, which should define the required level of access agreeable to both provider and consuming application. The foundation of access control is a provider granting or denying a consuming application and/or consumer access to a resource to a certain level of granularity.

Following user and application authorization strategies can be used for managing access control:

Groups/Role Based Access Control:

Role based Access Control (RBAC) represents a simple access control mechanism. The

consumers have a static function defined by the group boundaries. Grouping on basis of roles is purely business level decision. Identity Providers are responsible for retrieving this group information from the Identity Store. A role is an application specific definition of access control and a user is allowed to access an API based on the role and group s/he belongs to.

Policy/Attribute -based Access Control:

Instead of assigning static roles to consumers based on the groups they represent, Attribute-based Access Control(ABAC) aims to facilitate the dynamic determination of access control based on information available at the time of the API call. Things like the time of day, the role, the location of the API, the location of the App and combinations of conditions contribute to the determination of the degree of access. XACML is a standard, which defines the rules that should be executed in order to evaluate the level of access at the time of the API call.

Mechanism	Purpose/When to Use
Groups/ Role Based	Recommended for internal facing API and can be used for external as well
Policy/ Attribute based	Recommended for internal and external facing API

Best Practices for API Authorization

- ★ Based on the services (APIs) that are exposed, additional fine-grained access controls can be applied using scopes. For example, a data service might provide

“read” and “write” scopes.

- ★ Once the token is issued to a client application, the access rights bounded by the scope are encapsulated in the Access Token for the length of its validation period.
- ★ When the “Authorisation code” token is passed to the Authorisation Server, a scope parameter is included to define what scopes the client can use.
- ★ This is an important consideration as it can impact how the API service is defined e.g. single service with multiple scopes or multiple services with single scopes.

6.2.3 Confidentiality and Integrity

Confidentiality and integrity cover the handling of request and response data, both in transit and at rest. The aim is to protect the payload content from unauthorised access, manipulation or faking. An API request should be received intact by the API, with validation as to the source of the request. Untampered API responses need to be received by the consuming application, with confirmation that they are legitimately received from the API.

- ★ All communications to or from an API should be over TLS 1.2 or higher. Other versions of TLS and SSL should be disabled.
- ★ The consuming application should validate the TLS certificate chain when making requests to protected resources, including checking the Certificate

Revocation List (CRL) through Online Certificate Status Protocol (OCSP)

Content Encryption

If content needs only to be visible to specific consumer end points, use encryption. However, if content only should be guaranteed untampered and/or from a specific source (e.g. provider), then use content signing.

Whilst TLS protects the payload in transit, it only applies to each point-to-point connection between components (e.g. mobile app to API gateway). If transit components are not totally under the provider's control, it can be worthwhile performing body encryption despite it being communicated over TLS, which offers channel encryption. For example, it may be sensible to encrypt credit card details passed between consumer and provider backend systems.

Content Signing

Content signing is used to assure content integrity and proof of authorship. It can apply to the whole body of the message or specific elements of that content. For example, credit card details.

Best Practices for Confidentiality and Integrity

- ★ Ensure secure key management system for managing the keys.
- ★ Ensure standard cryptographic algorithm existing within coding libraries is used for performing the encryption and signing.
- ★ Signing has less of a computational overhead than encryption, but can still

affect the performance, so it is advisable that it be used only when and where needed.

- ★ Secure communication channel and client/server validation: Transport Level Security (TLS) is a standard approach to securing an HTTP channel for confidentiality, integrity, and authentication. With two-way TLS, client and server authenticate each other.
- ★ Enabling SSL/TLS encryption
- ★ Prevent API call tampering
- ★ Securing the keys

6.2.4 Non-repudiation

Non-repudiation covers the mechanisms that ensure that a consumer cannot deny making a request and, similarly, a provider cannot claim they did not send a response. To aid non-repudiation for APIs, it is important to ensure credentials are not shared between consumers and to perform comprehensive logging of API request/responses.

Digital signatures are useful for not just guaranteeing authenticity and integrity, but also supporting non-repudiation.

Best Practices for Non-repudiation

- ★ Secure handling of user credentials and sensitive data
- ★ Comprehensive logging of API requests and responses
- ★ Using digital signing and verification

Annex. III provides checklists on “API Security Vulnerabilities” and “API Security”

7. API from Standard Entities

UPI API

UPI API are set of APIs provided by NPCI, which allow transfer of money between account holders. Many apps such as BHIM, Tez, Phonepe, Paytm, SBI Pay, iMobile, chillr, etc., use UPI platform to transfer money.

S. No.	API Name	Description
1.	VPA Verification	This API is used to check whether a particular UPI ID is valid or not. It also fetches Customer name as registered in Account
2.	Merchant Collect	This API is used to create a money request from merchant to customer in the form of UPI collect notification/ request in the respective UPI app. There is an expiry time after which request expires automatically
3.	Check Transaction Status	This API is used to check the status of transaction whether it was successful or not with the response code. It works based on fetching status against Transaction ID
4.	Merchant Cash Back	This is a Payment API and will serve the purpose of sending money from merchant account to a customer account via UPI. It can be used for cashbacks, refunds, disbursements, etc.

(Source: <https://api.kotak.com/documentation/payments/upiwebcollect-doc>)

OpenBank API

The main idea behind the OpenBank API is to allow third party developers (Fintech) to build applications around financial institutions with increased financial transparency options for account holders to carry out their purchase, lending and investment activities. World over, UK, US, Germany, etc., shaped up the strategies around OpenBankAPI.

Payment services (PSD 2) (2015) - Directive (EU) 2015/2366

(Source: https://ec.europa.eu/info/law/payment-services-psd-2-directive-eu-2015-2366_en)

WebPayment API

Web Payments is an emerging web standard being developed by the W3C to simplify online payments and enable a broader set of players to participate easily in the payments ecosystem on the web. The standards are flexible; they work with various types of payment systems and are intended to work on any browser on any device, payment method, or payment service provider. This flexibility enables development simplicity, deployment consistency, and future compatibility with emerging payment technologies.

(Source: <https://developers.google.com/web/fundamentals/payments/>)

Annex: I

This section lists indicative guidelines or checklists that can be referred to while defining checklists within each organization for development, deployment and opening APIs for partners. These are samples to help banks define their own standards.

A. API Risk Evaluation Guideline

APIs can be categorized based on criticality

API Criticality	Description
High	These are the types of APIs where there is a debit or a credit request for the API. For example, NEFT, RTGS, UPI P2M & Collect, Customer data prefill, Any data for which bank owns IPR
Medium	These are the types of APIs where there is substantial customer data exchanged like Customer Account number, Cust ID, Aadhar No., PAN Card, Credit Card No., Debit Card No. For example, Lead Generation, OTP, SI Mandate
Low	These are typically enquiry APIs where there is no substantial customer data exchanged which is publicly available – Branch master fetch, ATM locator

API Characteristics Classification

API Characteristics	High	Medium	Low
API for depositing data in bank other than payments	All depositing request which has customer specific information and is end to end STP at bank	All depositing request is a drop on the floor at bank and will be further reviewed/action taken as per the existing process	
API for fetching information from bank	API Supports information which is pre-calculated by bank or data for which bank owns IPR Or API used for fetching information of the partner other than own account details	API Supports information like Cards/eKYC, Balance Statement etc., i.e, information which is sensitive	API Supports information like Masters which are not relevant to partner or generic in nature like list of branches, ATMs, etc.
API for financial transactions	Partner is originating payment request, which is STP at bank		

B. Security Recommendations Checklist

Security Policies to be Applied

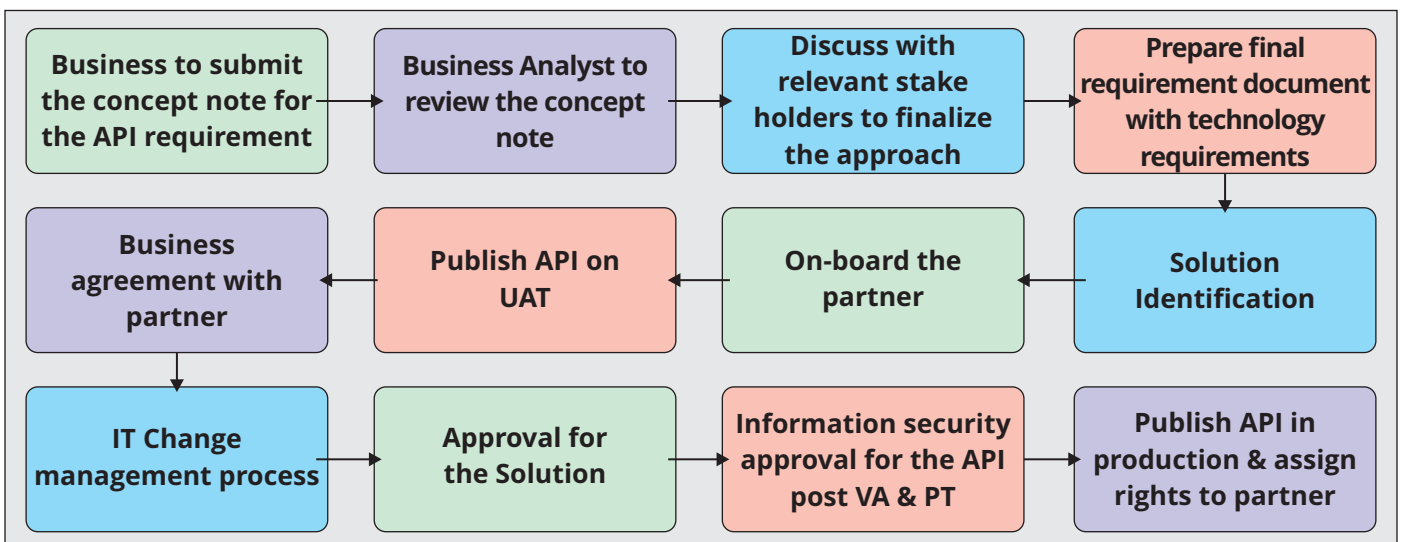
Activity	Where to Perform	Action
Authentication of API Client	Gateway	Transport Layer Security
	Producer System	Scope & Payload Validation
Quota Management & Throttling	Gateway	Configuration
Content scanning for threats such as SQL Injection	WAF Gateway	Certificate to be configured at WAF
Content validation (JSON schema, XML schema)	Gateway Producer System	Payload validation should be validated and reviewed in Producer system
Encryption of sensitive information	Gateway	Business Analyst team to review sensitive information during the design stage for each API
Automated attack/bot- detection	DDOS WAF Gateway	Gateway Certificate to be configured at WAF and Bot detection to be enabled at Gateway for each API
Message Level Security	Gateway	Encryption, JWT, etc.
Signature Validation		If encryption is enabled
API key management		API Key to be issued to all registered partners

Security Recommendations by API Risk Levels

S. No.	Security Parameter	Low Risk	Medium Risk	High Risk
1.	TLS	Mandatory	Mandatory	Mandatory
2.	TLS with Client Authentication	Optional	Mandatory	Mandatory
3.	HTTP Methods (GET, POST etc.)	Post	Post	Post
4.	IP Whitelisting	Optional	Optional	Mandatory

S. No.	Security Parameter	Low Risk	Medium Risk	High Risk
5.	Protect Against SQL Injection	Mandatory	Mandatory	Mandatory
6.	Protect Against Code Injection	Mandatory	Mandatory	Mandatory
7.	Protect Against Cross-Site Request Forgery	Mandatory	Mandatory	Mandatory
8.	Protect Against Document Structure Threats	Mandatory	Mandatory	Mandatory
9.	Schema Validation	Mandatory	Mandatory	Mandatory
10.	Rate Limit(DDOS)	Mandatory	Mandatory	Mandatory
11.	Limit Message Size	Mandatory	Mandatory	Mandatory
12.	Throughput Quota	Mandatory	Mandatory	Mandatory
13.	Authentication (HTTP basic, OAuth, SSO etc)	Optional	Optional	Mandatory
14.	Encryption	Optional	Mandatory	Mandatory
15.	Digital Signature Validation	Optional	Optional	Mandatory
16.	Limit Time Availability	Optional	Optional	Optional
17.	Content-Type	Optional	Mandatory	Mandatory
18.	File Size	Optional	Mandatory	Mandatory

C. Process for Publishing API



D. Approval Matrix for Publishing API

Approval process should be formulated on the basis of the organizational structure for allowing partners to access the APIs.

Illustrative Approval Matrix:

API Category	Business Head	IT Head	Info Sec Head	Operations Head	Risk
API for Depositing data in bank E.g.: Borrow API. Application deposit request & no prefill is involved & decision made by bank including credit under writing, Account Opening, Lead generation, etc.	Yes	Yes	Yes		
API for not fetching personally identifiable information, E.g.: OTP, Branch Master, Lead status update	Yes	Yes	Yes		
API for fetching customer specific information which includes KYC information, Cards, etc.	Yes	Yes	Yes	Yes	
API for fetching data which is banks IPR	Yes	Yes	Yes	Yes	Yes
API for Financial Transactions E.g. : Corporate Debit & Credit, Full Origination API like LOS, APS, SME , Bureau score, Income Score, etc.	Yes	Yes	Yes	Yes	Yes

Exception Approval process for publishing Open API to Partners

In case of any exceptions in the API security requirement, it is recommended to formulate a process for an exception approval.

Illustrative Exception Approval Matrix:

API Category	Business Head	IT Head	Info Sec Head	Operations Head	Risk/ Info. Security Head
API for Depositing data in bank E.g.: Borrow API Application deposit request & no prefill is involved & decision made by bank including credit under writing, Account Opening, Lead generation, etc	Yes	Yes	Yes		
API for not fetching personally identifiable information E.g.: OTP , Branch Master	Yes	Yes	Yes		
API for fetching customer specific information which includes KYC information, Cards, etc. E.g.: Dedupe API	Yes	Yes	Yes	Yes	
API for fetching data which is banks IPR Eg: Offer API	Yes	Yes	Yes	Yes	Yes
API for Financial Transactions E.g.: Corporate Debit & Credit, Full loan origination with eligibility calculation, Bureau score, etc	Yes	Yes	Yes	Yes	Yes

E. Partner Risk Evaluation

Parameter	High	Medium	Low
Relationship	Aggregator	Partner has hosted systems at partners premise and used by bank for processing or corporate depositing own data	Govt Business

Parameter	High	Medium	Low
Goodwill	Fraud or Governance Concerns; Current Financial Losses or Service Delivery Concerns	Historical News of Legal issues or Fraud	No knowledge of Fraud Legal Proceedings, Financial Losses or Service Delivery Concerns
Size	Small National Player e.g. Asset Size	Mid size National / Global Player e.g. Asset Size	Large National / Global Player e.g. Asset Size
Net Profit (simple average of last 3 years)	<5% of Revenue	>=5% and <=10% of Revenue	>10% of Revenue
Certifications (e.g. ISO27001, etc.) & PCIDSS Compliance	No Certifications done as per industry standards and not having membership of Professional Associations	1 or more certifications done and also have membership of few Professional Associations	Certified and compliant with all necessary Industry standards and membership of many Professional Associations
API Security Standards	More than 1 deviation in the standards recommended by bank	Only 1 deviation in the standards recommended by bank which is mutually agreed and approved by ISG	Agreeing to all standards recommended by bank completely as per bank standards

F. Partner On-boarding RACI

RACI Matrix for API Management

Team	Business	Business Analyst	IT	Information Security
Discussion with customer for their requirements	A	R	C	C
Contracts with Partner	A	R	C	C
Ensuring right API security	C	C	R	A
Onboarding of Partner	A	C	C	C
API Usage of the Partner & Monetizing	A	I	I	I

Annex: II

API Technology Management – Responsibilities Matrix

ROLES TO MANAGE THE API LIFECYCLE			
API Developer	Security Professional	Operations Engineer	API product owner/Business owner
▪ Service callouts ▪ XML to/from JSON ▪ Path validation, parameter validation, header validation ▪ Warnings, error logs, debug logs, info logs ▪ Thread pools, exception counts, object counts ▪ Round robin, least loaded, retries ▪ Fallbacks, back pressure, multiple implementations ▪ Caching ▪ Encryption, masking ▪ Custom usage tracking ▪ Custom usage records, custom limit enforcement ▪ Custom caller journey, custom user journey ▪ API access keys, database credentials, certificate keys ▪ Key value map, document store, graph, relational store	▪ Mutual authentication SSL, VPN, IP whitelisting ▪ API key validation ▪ API key authorization ▪ API key & request/response logging ▪ OAuth access token Validation ▪ OAuth scopes check ▪ User & request/response logging ▪ Bots, SQL injection, virus, compromised user, compromised APIkey ▪ Confidential data screening, PII data ▪ Screening, data masking ▪ SSL, storage encryption	▪ Caching ▪ Access logging, custom logging ▪ Latencies, status codes ▪ Rate limits, spike arrests, concurrency limits, quotas ▪ Round robin, least-loaded, Retries ▪ Retries, fallbacks, back pressure, multiple implementations ▪ Progressive rollout, experimentation	▪ Per-caller tracking, per-user tracking, ▪ Per-API tracking ▪ Subscription validation, limit enforcement ▪ Caller journey, user journey

Source: https://pages.apigee.com/rs/351-WXY-166/images/API-54308-ProductBrief_Checklist_V4%20%281%29.pdf

Annex: III

A. API Security Vulnerability List

General Security Threat/Vulnerability List

#	Vulnerability	Mitigation
1.	Broken Authentication - Permits Credentials stuffing - Permits Brute Force - Permits Weak Passwords - Use plain text, encrypted or weakly hashed password - Has missing or ineffective multi-factor authentication - Exposes session ID in url - Do not properly invalidate authentication tokens	- Strong password strength. - Efficient password uses and change controls management. - Secure password storage. - Protecting Credentials in Transit. - Session ID Protection. - Account lists. - Browser caching
2.	Sensitive Data Exposure - Old or weak crypto keys are used - Encryption is not enforced (HTTPS is not enforced) Insufficient SSL/TLS configuration - User agent does not verify if the received server certificate is valid or not	- Encrypt the data and define accessibility. - Secure authentication gateways. - Prevent password attacks. - Conduct regular risk assessment - Keeping a protected and secure backup of the sensitive data
3.	Broken Access Control - Bypassing access control checks by modifying the URL - Metadata manipulation, such as replaying or tampering a JSON Web Token (JWT) or a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation - Allowing the primary key to be changed to another users record, permitting viewing or editing someone else's account. - JWT insecurity (missing cryptographic signature or MAC)	- Protect and Limit (whitelist) the HTTP Methods (GET, PUT etc) exposed. - Validate Method(s) for session token / API key.

#	Vulnerability	Mitigation
3.	- Replaying or tampering with JSON Web Token - Abusing JWT invalidation - Replaying or tampering with Cookie - Force browsing to authenticated pages as an unauthenticated user or to privileged pages as a standard user. - Accessing API with missing access controls for POST, PUT and DELETE. - API keys revocation is not done - API key compromise - Exposure of inappropriate API methods to access services	
4.	Elevation of Privilege Acting as a user without logging in, or acting as an admin when logged in as a user.	- Define the authorization 'scope' that strictly adhere with the accessing of the resources. - Controlling egress of information via the API, aligned to set access permissions/policies.
5.	Cross Site Scripting <i>Reflected XSS</i>: The application or API includes invalidated and un-escaped user input as part of HTML output. A successful attacker can execute arbitrary HTML and JavaScript in the victim's browser. <i>Stored XSS</i>: The application or API stores non-sanitized user input that is viewed later by another user or an administrator. Stored XSS is a high or critical risk. <i>DOM XSS</i>: JavaScript frameworks, single-page applications, and APIs that dynamically include attacker-controllable data to a page are vulnerable to DOM XSS. Ideally, the application would not send attacker-controllable data to unsafe JavaScript APIs	- Validate Input - Message analysis to block parameter attacks such as cross-site scripting.
6.	Cross-Site Request Forgery	Use token with state and nonce parameters.

#	Vulnerability	Mitigation
7.	Parameter Tampering Malicious Input, Injection attacks and Fuzzing ▪ HTTP Verb Tampering ▪ Insecure Direct Object Reference ▪ Unvalidated redirects and forwards	▪ Validate input: Secure parsing and strong typing. ▪ Validate incoming content-type application/json. ▪ Validate JSON content. ▪ Validate XML (schema and format). ▪ Scan attachments. ▪ Produce valid HTTP Return Code. ▪ Validate response type.
8.	Man-in-the-middle Attack ▪ Un encrypted channels and unsigned and verified data can lead to man-in-the middle attacks	▪ Always use encrypted channels ▪ Digitally sign and verify the data ▪ Authenticate the source and verify transaction through 2nd factor and out-of-band channel
9.	Denial Of Service Attack	▪ Throttle access to all exposed APIs. ▪ Monitor use to indicate possible DoS attacks
10.	Injection Attack ▪ SQL Injection ▪ No SQL Injection ▪ LDAP injection	▪ Message analysis to block parameter attacks such as SQL injection, No SQL injection, and LDAP injection

B. API Security Assurance Checklist

#	Security Requirements	Assurance Checklist
1.	Authentication	▪ Verify that every API endpoint requires authentication by default unless it is intended to be public ▪ Enforce all authentication controls on the server side. ▪ Enforce all authentication controls to fail securely. ▪ Ensure that account passwords are one way hashed with a salt, to defeat brute force and password hash recovery attacks. ▪ Ensure that credentials such as passwords, security tokens and API keys don't appear in URL. ▪ Ensure others cannot access API keys and are always transmitted and saved securely.

#	Security Requirements	Assurance checklist
2.	Authorization (Access Control)	- Ensure that data is accessible to each authorized user alone. - Verify that access controls fail securely. - Verify that the access control rules match across different layers of API communication i.e API client, middleware and provider. - Verify that all access control decisions and all failed decisions are logged.
3.	Confidentiality and Integrity (Data Protection)	- Verify that all sensitive data is sent to the server in the HTTP message body or headers (i.e., URL parameters are never used to send sensitive data). - Verify that the data processed by the API is identified, proper access control policy applied, encrypted and relevant data protection directives applied. - Verify that the application sets appropriate anti-caching headers as per the risk of the application, such as the following: Expires: Tue, 03 Jul 2001 06:00:00 GMT Last-Modified: now GMT CacheControl: no-store, no-cache, mustrevalidate, max-age=0 CacheControl: post-check=0, pre-check=0 Pragma: no-cache - Verify that data stored in client side storage (such as HTML5 local storage, session storage, IndexedDB, regular cookies or Flash cookies) does not contain sensitive data or personal identification information. - Verify that access of sensitive data is logged, if the data is collected under relevant data protection directives or where logging of accesses is required. - Verify that sensitive information maintained in memory is overwritten with zeros as soon as it is no longer required, to mitigate memory dumping attacks. Communication Security: - Verify that each server certificate in the path of communication is valid right from the trusted root CA through the intermediated CAs. - Verify that TLS is used for all connections (including both external and backend connections) that involve sensitive data or functions, and does not fall back to insecure or unencrypted protocols. Ensure the strongest alternative is the preferred algorithm. - Verify that backend TLS connection failures are logged - Verify that certificate paths are built and verified for all client certificates using configured trust anchors and revocation information. - Verify that TLS certificate public key pinning (HPKP) is implemented with production and backup public keys. - Verify that HTTP Strict Transport Security headers are included on all requests and for all subdomains, such as Strict-TransportSecurity: max-age=15724800; includeSubdomains

#	Security Requirements	Assurance Checklist
4.	HTTP Security	- Verify that the application accepts only a defined set of required HTTP request methods. - Verify that every HTTP response contains a content type header specifying a safe character set (e.g., UTF-8, ISO 8859-1). - Verify that the HTTP headers or any part of the HTTP response do not expose detailed version information of system components. - Verify that the same encoding style is used between the client and the server. - Verify that XML or JSON schema is in place and verified before accepting input. - Verify that there are security mechanisms against Cross-site Scripting.
5.	Error Handling	- Verify that the application does not output error messages or stack traces containing sensitive data that could assist an attacker, such as session ID, software/framework versions and personal information - Verify that error handling logic in security controls denies access by default. - Verify that security logs are protected from unauthorized access and modification. - Verify that the API does not log sensitive data such as sensitive authentication data that could assist an attacker, including user's session identifiers, passwords, hashes, or API tokens.

C. Token Threat/Vulnerability and Mitigation

JSON Web Token (JWT) is a compact token format intended for space consideration environments such as HTTP authorization headers and URI query parameters. JWT consists of three different elements: the header, the payload, and the signature/encryption data. The first two elements are JSON objects of a certain structure. The third is dependent on the algorithm used for signing or encryption, and, in the case of unencrypted JWTs, it is omitted. Deciding the JWT parameter for implementing the security measures depends on the following:

Access Token: This token is used to allow the service to access a requesting service via API interface and used by the client application to access protected resources on the provider, and it can be signed. It is a random character string that also contains "scope" information to allow additional access policies to be applied. E.g.: duration of access. The uses of access token consists of following set of rules:

- ★ It is required that the token be protected both in transit (TLS) and in storage (encryption)

- ★ The “Authorisation Request Header Field” format is required to only be used to transmit tokens
- ★ It is Recommended that the life time of this token be set to 60 mins
- ★ It is Recommended that scopes be used for coarse and fine grained access
- ★ Form-Encoded Body Parameter is Not Recommended for transmission
- ★ URI Query Parameter is Not Recommended for transmission

Refresh Token: Used to obtain new Access tokens when the old one expires or is invalid. The uses of refresh token consist of following set of rules:

- ★ It is Required that the token be protected both in transit (TLS) and in storage (encryption)
- ★ It is Recommended that the lifetime of this token be set to 24 hours

#	Threat/Vulnerability	Mitigation
1.	Malicious token generation and modification by man in the middle attack	▪ Digital signing of tokens (e.g. JWS with JWT) or attaching a Message Authentication Code (MAC) ▪ Verify JWT (Json) tokens are handled in a secure way. ▪ Verify JWT (Json) tokens are invalidated after logout. ▪ Verify JWT in cookies is only accessible on the user domain.
2.	Token Disclosure ▪ Man-in-the-middle attack ▪ The Access Token is passed in clear text with no hashing, signing or encryption.	Communication Security Use TLS 1.2 with a cipher suite that includes DHE or ECDHE. ▪ The client application should validate: • The TLS certificate chain • Stored the certificate revocation list in secure manner • Check the certificate revocation list
3.	Token Redirects Ensure the Authentication and Resource Servers are “paired” and the access token can only be used between the specified servers	▪ Using the “audience” header (defined currently in a draft RFC), the client application, resource server and authorisation server can help ensure that the token can only be used on the resource servers requested by the client and recognised by the authorisation server. ▪ Also addressed with “state” parameter in the header ▪ Signing of tokens is also applicable to address token redirects
4.	Token Replay The malicious actor copies an existing token (e.g. refresh token or authorisation code) and reuses it with his or her own request.	▪ Define Limit lifetime of the token in such a way that it turns it into a short lived issue. ▪ Use signed requests along with nonce and timestamps. ▪ Validate TLS certificate chain when accessing Resource Server

Glossary

#	Acronym	Full Form
1	ABAC	Attribute Based Access Control
2	API	Application Programming Interface
3	APS	Automated Payment System
4	B2B	Business to Business
5	BFSI	Banking and Financial Services Industry
6	BLA	Business Logic Attacks
7	CRL	Certificate Revocation List
8	DB	Data Base
9	DDOS	Distributed Denial of Service
10	DHE	Diffie-Hellman Ephemeral
11	DOM	Document Object Model
12	DoS	Denial of Service
13	ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
14	GET	Get Method
15	GMT	Greenwich Mean Time
16	HPKP	HTTP Public Key Pinning
17	HPP	HTTP Parameter Pollution
18	HTML	Hyper Text Markup Language
19	HTTP	Hyper Text Transport Protocol
20	IP	Internet Protocol
21	IPR	Intellectual Property right
22	ISG	Information Security Group
23	JSON	Java Script Object Notation
24	JWT	Java Web Token
25	KYC	Know Your Customer
26	LDAP	Light Weight Directory Access Protocol
27	LOS	Loan Origination System
28	MAC	Message Authentication Code
29	MFA	Multi Factor Authentication
30	NEFT	National Electronic Fund Transfer
31	NPCI	National Payment Corporation of India
32	OCSP	Online Certificate Status Protocol
33	OTP	One Time Password
34	OWASP	Open Web Application Security Project
35	PCIDSS	Payment Card Industry, Data Security Standard

#	Acronym	Full Form
36	PEP	Policy Enforcement Point
37	PII	Personally Identifiable Information
38	PKI	Public Key Infrastructure
39	POST	HTTP Post method
40	PSD2	Payment Services Directive 2
41	PUT	Put method
42	QoS	Quality of Service
43	RACI	Responsible, Accountable, Consulted, Informed
44	RBAC	Role Based Access Control
45	RFC	Request for Comment
46	RTGS	Real Time Gross Settlement
47	SAML	Security Assertion markup Language
48	SLA	Service Level Agreements
49	SME	Small and Medium Enterprise
50	SQL	Structured Query Language
51	SSL	Secure Socket Layer
52	SSO	Single Sign On
53	STP	Straight Through Processing
54	TLS	Transport Level Security
55	TTP	Trusted Third Party
56	UAT	User Acceptance Test
57	UI	User Interface
58	UPI	Unified Payment Interface
59	URI	Uniform Resource Indicator
60	URL	Uniform Resource Locator
61	VAPT	Vulnerability Assessment and Penetration Testing
62	VPA	Virtual Payment Address
63	VPN	Virtual Private Network
64	W3C	World Wide Web Council
65	WAF	Web Application Firewall
66	XACML	XML based Access Control Markup Language
67	XML	Extensible Markup Language
68	XSS	Cross Site Scripting Attack

References

- [1] https://www.owasp.org/index.php/Top_10-2017_A2-Broken_Authentication
- [2] <https://www.mulesoft.com/resources/api-strategy>
- [3] <https://www.softwaretestinghelp.com/api-management-tools/>
- [4] The business of APIs best practices IBM Developer
<https://developer.ibm.com/apiconnect/resources/the-business-of-apis-best-practices/>
- [5] <https://api.kotak.com/documentation/payments/upiwebcollect-doc>
- [6] Payment services (PSD 2) (2015) - Directive (EU) 2015/2366 -
https://ec.europa.eu/info/law/payment-services-psd-2-directive-eu-2015-2366_en
- [7] <https://developers.google.com/web/fundamentals/payments/>
- [8] https://pages.apigee.com/rs/351-WXY-166/images/API-54308-ProductBrief_Checklist_V4%20%281%29.pdf
- [9] <https://security.uci.edu/AppSecChecklist-V1.0.doc>
- [10] API-Standard-and-Guidelines-Part-A-Business-v1.0-October-2016
<https://www.ict.govt.nz/guidance-and-resources/standards-compliance/api-standard-and-guidelines/api-standard-and-guidelines-part-a-business/>
- [11] API-Standard-and-Guidelines-Part-B-Technical-v1.0-October-2016:
<https://www.ict.govt.nz/guidance-and-resources/standards-compliance/api-standard-and-guidelines/api-standard-and-guidelines-part-b-technical/>
- [12] Error Handling good practices: <https://docs.microsoft.com/en-us/azure/api-management/api-management-error-handling-policies>
- [13] Design guidance to APIs: <https://docs.microsoft.com/en-us/azure/architecture/best-practices/api-design?toc=%2Fazure%2Fapi-management%2Ftoc.json>
- [14] Good practice for API Implementation: <https://docs.microsoft.com/en-us/azure/architecture/best-practices/api-implementation?toc=%2Fazure%2Fapi-management%2Ftoc.json>
- [15] Example of deploying API management across multiple Data centers:
<https://docs.microsoft.com/en-us/azure/api-management/api-management-howto-deploy-multi-region>

CONTRIBUTORS

Mentor

DR. A.S. RAMASASTRI, Director, IDRBT

Members

Shri GopaKumar G, Senior Vice President, HDFC bank

Shri Yash Shah, Axis ESB Team, Axis Bank

Shri Dhiraj Ranka, Information Risk Management Team, Kotak Mahindra Bank

Shri Saran K Joseph, Deputy Vice President, Federal Bank

Shri Amit Ranjan, Integration Specialist, IBM

Shri Keshri Asthana, Cloud Solution Architect Microsoft

Shri Senthil Doraiswamy, Solution Architect, Google

Shri Vivek Srivastava, SVP Research and Innovation, ReBIT

Shri Susanta Dash, Assistant Vice President, ReBIT

Shri Deep Narayan Tiwari, Manager, ReBIT

Shri P. Parthasarathi, Chief Technology Officer, IDRBT

Dr. V. Radha, Associate Professor, IDRBT

Shri Lalit Mohan, Senior Domain Expert, IDRBT

Ms. Mrudula Petluri, Senior Domain Expert, IDRBT



Institute for Development and Research in Banking Technology
(Established by Reserve Bank of India)

Castle Hills, Road No.1, Masab Tank, Hyderabad - 57.
EPABX: +91 40 2329 4999, Fax: +91 40 2353 5157
Web: www.idrbt.ac.in, e-mail: publisher@idrbt.ac.in